

Due: Wednesday, May 1 at 11:59 pm

Deliverables:

1. Submit a PDF of your homework, **with an appendix listing all your code**, to the Gradescope assignment entitled “Homework 7 Write-Up”. In addition, please include, as your solutions to each coding problem, the specific subset of code relevant to that part of the problem. You may typeset your homework in LaTeX or Word (submit PDF format, **not** .doc/.docx format) or submit neatly handwritten and scanned solutions. **Please start each question on a new page.** If there are graphs, include those graphs in the correct sections. **Do not** put them in an appendix. We need each solution to be self-contained on pages of its own.
 - In your write-up, please state with whom you worked on the homework.
 - In your write-up, please copy the following statement and sign your signature next to it. (Mac Preview and FoxIt PDF Reader, among others, have tools to let you sign a PDF file.) We want to make it *extra* clear so that no one inadvertently cheats.

“I certify that all solutions are entirely in my own words and that I have not looked at another student’s solutions. I have given credit to all external sources I consulted.”
2. Submit all the code needed to reproduce your results to the Gradescope assignment entitled “Homework 7 Code”. Yes, you must submit your code twice: once in your PDF write-up following the directions as described above so the readers can easily read it, and once in compilable/interpretable form so the readers can easily run it. Do **NOT** include any data files we provided. Please include a short file named README listing your name, student ID, and instructions on how to reproduce your results. Please take care that your code doesn’t take up inordinate amounts of time or memory to run. If your code cannot be executed, your solution cannot be verified.

1 Honor Code

Declare and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature : _____

While discussions are encouraged, *everything* in your solution must be your (and only your) creation. Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

2 The Training Error of AdaBoost

Recall that in AdaBoost, our input is an $n \times d$ design matrix X with n labels $y_i = \pm 1$, and at the end of iteration T the importance of each sample is reweighted as

$$w_i^{(T+1)} = w_i^{(T)} \exp(-\beta_T y_i G_T(X_i)), \quad \text{where} \quad \beta_T = \frac{1}{2} \ln \left(\frac{1 - \text{err}_T}{\text{err}_T} \right) \quad \text{and} \quad \text{err}_T = \frac{\sum_{y_i \neq G_T(X_i)} w_i^{(T)}}{\sum_{i=1}^n w_i^{(T)}}.$$

Note that err_T is the weighted error rate of the classifier G_T . Recall that $G_T(z)$ is ± 1 for all points z , but the metalearner has a non-binary decision function $M(z) = \sum_{t=1}^T \beta_t G_t(z)$. To classify a test point z , we calculate $M(z)$ and return its sign.

In this problem we will prove that if every learner G_t achieves 51% accuracy (that is, only slightly above random), AdaBoost will converge to zero training error. (If you get stuck on one part, move on; all five parts below can be done without solving the other parts, and parts (c) and (e) are the easiest.)

- (a) We want to change the update rule to “normalize” the weights so that each iteration’s weights sum to 1; that is, $\sum_{i=1}^n w_i^{(T+1)} = 1$. That way, we can treat the weights as a discrete probability distribution over the sample points. Hence we rewrite the update rule in the form

$$w_i^{(T+1)} = \frac{w_i^{(T)} \exp(-\beta_T y_i G_T(X_i))}{Z_T} \tag{1}$$

for some scalar Z_T . Show that if $\sum_{i=1}^n w_i^{(T)} = 1$ and $\sum_{i=1}^n w_i^{(T+1)} = 1$, then

$$Z_T = 2 \sqrt{\text{err}_T (1 - \text{err}_T)}. \tag{2}$$

Hint: sum over both sides of (1), then split the right summation into misclassified points and correctly classified points.

- (b) The initial weights are $w_1^{(1)} = w_2^{(1)} = \dots = w_n^{(1)} = \frac{1}{n}$. Show that

$$w_i^{(T+1)} = \frac{1}{n \prod_{t=1}^T Z_t} e^{-y_i M(X_i)}. \tag{3}$$

- (c) Let B (for “bad”) be the number of sample points out of n that the metalearner classifies incorrectly. Show that

$$\sum_{i=1}^n e^{-y_i M(X_i)} \geq B. \tag{4}$$

Hint: split the summation into misclassified points and correctly classified points.

- (d) Use the formulas (2), (3), and (4) to show that if $\text{err}_t \leq 0.49$ for every learner G_t , then $B \rightarrow 0$ as $T \rightarrow \infty$.
Hint: (2) implies that every $Z_t < 0.9998$. How can you combine this fact with (3) and (4)?
- (e) Explain briefly why AdaBoost with short decision trees is a form of subset selection when the number of features is large.

$$(a) \quad \sum_{i=1}^n w_i^{(T+1)} = \sum_{i=1}^n \frac{w_i^{(T)} \exp(-\beta_T y_i G_T(x_i))}{Z_T} = \frac{1}{Z_T} \sum_{i=1}^n w_i^{(T)} \exp(-\beta_T y_i G_T(x_i))$$

$$\begin{aligned} \sum_{i=1}^n w_i^{(T)} \exp(-\beta_T y_i G_T(x_i)) &= \sum_{i: y_i \neq G_T(x_i)} w_i^{(T)} e^{\beta_T} + \sum_{i: y_i = G_T(x_i)} w_i^{(T)} e^{-\beta_T} \\ &= \left(\sum_{i=1}^n w_i^{(T)} \right) e^{-\beta_T} + (e^{\beta_T} - e^{-\beta_T}) \sum_{i: y_i \neq G_T(x_i)} w_i^{(T)} \\ &= e^{-\beta_T} + (e^{\beta_T} - e^{-\beta_T}) \left(\sum_{i: y_i \neq G_T(x_i)} w_i^{(T)} / \sum_{i=1}^n w_i^{(T)} \right) \\ &= e^{-\beta_T} + (e^{\beta_T} - e^{-\beta_T}) \text{err}_T \end{aligned}$$

$$e^{\beta_T} = \sqrt{\frac{1 - \text{err}_T}{\text{err}_T}} \quad e^{-\beta_T} = \sqrt{\frac{\text{err}_T}{1 - \text{err}_T}}$$

$$\Rightarrow Z_T = \sqrt{\frac{\text{err}_T}{1 - \text{err}_T}} + \left(\sqrt{\frac{1 - \text{err}_T}{\text{err}_T}} - \sqrt{\frac{\text{err}_T}{1 - \text{err}_T}} \right) \text{err}_T = 2 \sqrt{\text{err}_T (1 - \text{err}_T)}$$

$$(b) \quad w_1^{(1)} = w_2^{(1)} = \dots = w_n^{(1)} = \frac{1}{n}$$

$$w_i^{(2)} = \frac{1}{Z_1} w_i^{(1)} \exp(-\beta_1 y_i G_1(x_i)) = \frac{1}{n} \frac{1}{Z_1} \exp(-\beta_1 y_i G_1(x_i))$$

$$\begin{aligned} w_i^{(3)} &= \frac{1}{n} \cdot \frac{1}{Z_1 \cdot Z_2} \cdot \exp(-\beta_1 y_i G_1(x_i)) \cdot \exp(-\beta_2 y_i G_2(x_i)) \\ &= \frac{1}{n} \frac{1}{\prod_{t=1}^2 Z_t} \exp\left(-y_i \cdot \sum_{t=1}^2 \beta_t G_t(x_i)\right) \end{aligned}$$

$$\text{Assume } w_i^{(T)} = \frac{1}{n} \frac{1}{\prod_{t=1}^{T-1} Z_t} \exp\left(-y_i \sum_{t=1}^{T-1} \beta_t G_t(x_i)\right)$$

$$\begin{aligned} \text{Then } w_i^{(T+1)} &= \frac{1}{Z_T} w_i^{(T)} \exp(-\beta_T y_i G_T(x_i)) \\ &= \frac{1}{n} \frac{1}{\prod_{t=1}^T Z_t} \exp\left(-y_i \sum_{t=1}^T \beta_t G_t(x_i)\right) \end{aligned}$$

Therefore $w_i^{(T+1)}$ holds.

$$w_i^{(T+1)} = \frac{1}{n} \frac{1}{\prod_{t=1}^T Z_t} \exp\left(-y_i \sum_{t=1}^T \beta_t G_t(x_i)\right) = \frac{1}{n} \frac{1}{\prod_{t=1}^T Z_t} \exp(-y_i M(x_i))$$

(c) B is the number of sample points out of n that metalearner misclassifies.

$$\Rightarrow B = \sum_{x_i: \text{sgn}(M(x_i)) \neq y_i} 1 = \# \{i: y_i M(x_i) \leq 0\}$$

$$\begin{aligned} \sum_{i=1}^n \exp(-y_i M(x_i)) &= \left(\sum_{y_i M(x_i) > 0} + \sum_{y_i M(x_i) \leq 0} \right) \exp(-y_i M(x_i)) \geq \sum_{y_i M(x_i) \leq 0} \exp(-y_i M(x_i)) \\ &\geq \sum_{y_i M(x_i) \leq 0} 1 = B \end{aligned}$$

$$(d) \quad 1 = \sum_{i=1}^n w_i^{(T+1)} = \sum_{i=1}^n \frac{1}{n \prod_{t=1}^T Z_t} e^{-y_i \cdot M(x_i)} = \frac{1}{n} \frac{1}{\prod_{t=1}^T Z_t} \sum_{i=1}^n e^{-y_i \cdot M(x_i)} \geq B \cdot \frac{1}{n} \frac{1}{\prod_{t=1}^T Z_t}$$

$$\Rightarrow n \cdot \prod_{t=1}^T Z_t \geq B$$

$$\text{if } \text{err}_t \leq 0.49 \quad Z_t = 2 \sqrt{\text{err}_t (1 - \text{err}_t)} < 0.9998$$

$$B \leq n \cdot \prod_{t=1}^T Z_t < n \cdot (0.9998)^T \rightarrow 0 \quad \text{when } T \rightarrow \infty$$

(e) Each short tree uses only a few features. During training AdaBoost repeatedly picks trees that reduces weighted error. so features that help are chosen repeatedly while irrelevant ones are never used. Hence with many features and short trees the final model effectively depends only on the union of a small number of selected features. i.e. subset selection.

3 Movie Recommender System

In this problem, we will build a personalized movie recommender system! Suppose that there are $m = 100$ movies and $n = 24,983$ users in total, and each user has watched and rated a subset of the m movies. Our goal is to recommend more movies for each user given their preferences.

Our historical ratings dataset is given by a matrix $R \in \mathbb{R}^{n \times m}$, where R_{ij} represents the rating that user i gave movie j . The rating is a real number in the range $[-10, 10]$: a higher value indicates that the user was more satisfied with that movie. If user i did not rate movie j , $R_{ij} = \text{NaN}$.

The provided `movie_data/` directory contains the following files:

- `movie_train.mat` contains the training data, i.e. the matrix R of historical ratings specified above.
- `movie_validate.txt` contains user-movie pairs that don't appear in the training set (i.e. $R_{ij} = \text{NaN}$). Each line takes the form “ i, j, s ”, where i is the user index, j is the movie index, and s indicates the user's rating of the movie. Contrary to the training set, the rating here is binary: if the user liked the movie (positive rating), $s = 1$, and if the user did not like the movie (negative rating), $s = -1$.

We also provide `movie_recommender.py`, containing starter code for building your recommender system.

The singular value decomposition (SVD) is a powerful tool to decompose and analyze matrices. In lecture, we saw that the SVD can be used to efficiently compute the principal coordinates of a data matrix for PCA. Here, we will see that SVD can also produce dense, compact featurizations of the variables in the input matrix (in our case, the m movies and n users). This application of SVD is known as Latent Semantic Analysis (Wikipedia), and we can use it to construct a Latent Factor Model (LFM) for personalized recommendation.

Specifically, we want to learn a feature vector $x_i \in \mathbb{R}^d$ for user i and a feature vector $y_j \in \mathbb{R}^d$ for movie j such that the inner product $x_i \cdot y_j$ approximates the rating R_{ij} that user i would give movie j .

- Recall the SVD definition for a matrix $R \in \mathbb{R}^{n \times m}$ from [Lecture 21](#): $R = UDV^T$. Write an expression for R_{ij} , user i 's rating for movie j , in terms of only the contents of U , D , and V .
- Based on your answer above, what should we choose as our user and movie feature vector representations x_i and y_j to achieve 100% training accuracy (correctly predict all known ratings in R)?
- In the provided `movie_recommender.py`, complete the code for part (c) by filling in the missing parts of the function `svd_lfm`. Start by replacing all missing (NaN) values in R with 0. Then, compute the SVD of the resulting matrix, and follow your above derivations to compute the feature vector representations for each user and movie. Note: do **not** center the data matrix; this is not PCA.

Once you are finished with the code, the **rows** of the `user_vecs` array should contain the feature vectors for users (so the i th row of `user_vecs` is x_i), and the **rows** of `movie_vecs` should contain the feature vectors for movies (so the j th row of `movie_vecs` is y_j).

Hint: we recommend using `scipy.linalg.svd` to compute the SVD, with `full_matrices = False`. This returns U ($n \times m$), D (as a vector of m singular values in descending order, **not** a diagonal matrix), and V^T ($m \times m$) in that order.

- To measure the training performance of the model, we can use the mean squared error (MSE) loss,

$$\text{MSE} = \sum_{(i,j) \in S} (x_i \cdot y_j - R_{ij})^2 \quad \text{where } S := \{(i, j) : R_{ij} \neq \text{NaN}\}.$$

Complete the code to implement the training MSE computation within the function `get_train_mse`.

- (e) Our model as constructed may achieve 100% training accuracy, but it is prone to overfitting. Instead, we would like to use lower-dimensional representations for x_i and y_j to approximate our known ratings closely while still generalizing well to unknown user/movie combinations. Specifically, we want each x_i and y_j to be d -dimensional for some $d < m$, such that only the top d features are used to make predictions $x_i \cdot y_j$. The “top d features” are those corresponding to the d largest singular values: use this as a hint for how to prune your current user/movie vector representations to d dimensions.

In your code, compute pruned user/movie vector representations with $d = 2, 5, 10, 20$. Then, for each setting, compute the training MSE (using the function you implemented in part (d)), the training accuracy (using the provided `get_train_acc`), and the validation accuracy (using the provided `get_val_acc`). Plot the training MSE as a function of d on one plot, and the training and validation accuracies as a function of d together on a separate plot. The code for this part is already included in the starter code, so if your training MSE function from part (d) is implemented correctly, the required plots should be saved to your project directory.

Comment on which value of d leads to optimal performance.

Hint: as a sanity check, if implemented correctly, your best validation accuracy should be about 71%.

- (f) For sparse data, replacing all missing values with zero, as we did in part (c), is not a very satisfying solution. A missing value in the training matrix R means that the user has not watched the movie; this does not imply that the rating should be zero. Instead, we can learn our user/movie vector representations by minimizing the MSE loss, which only incorporates the loss on rated movies ($R_{ij} \neq \text{NaN}$).

Let's define a loss function

$$L(\{x_i\}, \{y_j\}) = \sum_{(i,j) \in S} (x_i \cdot y_j - R_{ij})^2 + \sum_{i=1}^n \|x_i\|_2^2 + \sum_{j=1}^m \|y_j\|_2^2$$

where S has the same definition as in the MSE. This is similar to the original MSE loss, except with two additional regularization terms to prevent the norms of the user/movie vectors from getting too large.

Implement an algorithm to learn vector representations of dimension d , the optimal value you found in part (e), for users and movies by minimizing $L(\{x_i\}, \{y_j\})$.

We suggest employing an alternating minimization scheme. First, randomly initialize x_i and y_j for all i, j . Then, minimize the above loss function with respect to the x_i by treating the y_j as constant vectors, and subsequently minimize the loss with respect to the y_j by treating the x_i as constant vectors. Repeat these two steps for a number of iterations. Note that when one of the x_i or y_j are constant, minimizing the loss function with respect to the other component has a closed-form solution. **Derive this solution first in your report, showing all your work.**

The starter code provides a template for this algorithm. Start by inputting your best d value from part (e) to initialize the user and movie vectors, and then implement the functions to update the user and movie vectors (holding the other constant) to their loss-minimizing values.

- To improve efficiency, we recommend using the `userRatedIdxs` and `movieRatedIdxs` arrays provided, which contain the indices of movies that each user rated and the indices of users that rated each movie (respectively), to iterate through the non-NaN values of R in the update functions.
- Run these 2 update steps for 20 iterations. Include your final training MSE, training accuracy, and validation accuracy on your report, and compare these results with your best results from part (e).

$$(a) \quad R = UDV^T \Rightarrow R_{ij} = (UDV^T)_{ij} = \sum_k \sum_q U_{ik} \cdot D_{kq} (V^T)_{qj}$$

$$D \text{ is diagonal} \Rightarrow D_{kq} = \begin{cases} \delta_k & k=q \\ 0 & k \neq q \end{cases}$$

$$\Rightarrow R_{ij} = \sum_k U_{ik} \cdot \delta_k \cdot V_{jk}$$

$$(b) \quad \text{let } U'_i = \begin{bmatrix} U_{i1} \cdot \delta_1 \\ U_{i2} \cdot \delta_2 \\ \vdots \\ U_{id} \cdot \delta_d \end{bmatrix}$$

$$\Rightarrow R_{ij} = \sum_k U'_{ik} \cdot V_{kj}^T$$

which means we can use

1. column i of U' as x_i

2. row j of V as y_j

so that $R_{ij} = x_i \cdot y_j$ inner product.

(c) $d=10$ leads to optimal performance. $\rightarrow$ val acc $\sim 72\%$

$$(f) \quad \mathcal{L}(\{x_i\}, \{y_j\}) = \sum_{(i,j) \in S} (x_i \cdot y_j - R_{ij})^2 + \sum_{i=1}^n \|x_i\|^2 + \sum_{j=1}^m \|y_j\|^2$$

First we treat y_j as constant.

$$\begin{aligned} 0 = \frac{\partial}{\partial x_i} \mathcal{L}(\{x_i\}, \{y_j\}) &= \frac{\partial}{\partial x_i} \left[\sum_{(i,j) \in S} (x_i \cdot y_j - R_{ij})^2 + \sum_{i=1}^n \|x_i\|^2 + \sum_{j=1}^m \|y_j\|^2 \right] \\ &= \sum_{j: (i,j) \in S} 2(x_i \cdot y_j - R_{ij}) y_j + 2x_i \\ &= \left(\sum_{j: (i,j) \in S} 2y_j y_j^T \right) \cdot x_i + 2x_i - \sum_{j: (i,j) \in S} 2R_{ij} y_j \end{aligned}$$

$$\Rightarrow x_i = \left(I + \sum_{j: (i,j) \in S} y_j^T y_j \right)^{-1} \sum_{j: (i,j) \in S} R_{ij} y_j$$

Similar to x_i , it's easy to get y_j as

$$y_j = \left(I + \sum_{i: (i,j) \in S} x_i^T x_i \right)^{-1} \sum_{i: (i,j) \in S} x_i R_{ij}$$

4 IM2SPAIN: Nearest Neighbors for Geo-location

For this problem, we will use nearest neighbors (NN or k -NN) to predict latitude and longitude coordinates of images from their CLIP embeddings. You'll be modifying starter code in the provided `im2spain` directory.

We are using a dataset of images scraped from Flickr with geo-tagged locations within Spain. Each image has been processed with OpenAI's CLIP image model (<https://github.com/openai/CLIP>) to produce features that can be used with k -NN.

The CLIP model was not explicitly trained to predict coordinates from images, but from task-agnostic pre-training on a large web-crawl dataset of captioned images has learned a generally useful mapping from images to embedding vectors. These feature vectors turn out to encode various pieces of information about the image content such as object categories, textures, 3D shapes, etc. In fact, these very same features were used to filter out indoor images from outdoor images in the construction of this dataset.

Note: Throughout the problem we use MDE which stands for Mean Displacement Error (in miles). Displacement is the (technically spherical) distance between the predicted coordinates and ground truth coordinates. Since all our images are located within a relatively small region of the globe, we can approximate spherical distances with Euclidean distances by treating latitude/longitude as cartesian coordinates. Assume 1 degree latitude is equal to 69 miles and 1 degree longitude is 52 miles in this problem.

Deliverables: Include your modified `im2spain.starter.py` script in your submission. Your submitted file should include all modifications requested in this problem.

- (a) Let's visualize the data. Using matplotlib and scikit-learn, plot the image locations and modify the code to apply PCA to the image features (remember to re-center the features first) in the `plot_data` method of `im2spain.starter.py`. Plot the data in its first two PCA dimensions, colored by longitude coordinate (east-west position).
- (b) Modify the starter code in `im2spain.starter.py` to find the three nearest neighbors in the training set of the test image file `53633239060.jpg`. Include those three image files (as images) in order from nearest to 3rd nearest in your submission. Now look at their coordinates. How many of the 3 nearest neighbors are "correct"?

Note: most images have been replaced with a placeholder image in the interest of storage space.



- (c) Before we begin with our k -NN model, let's first establish a naive constant baseline of simply predicting the training set centroid (coordinate-wise average) location for every test image. Modify the code in `im2spain.starter.py` to implement the constant baseline. What is its MDE in miles?
 $MDE = 209.9 \text{ miles}$
- (d) The main hyperparameter of a k -nearest neighbor classifier is k itself. Use a 1-D grid search in `im2spain.starter.py` to create a plot of the MDE (in miles) of k -NN regression versus k , where k is the number of neighbors. Include your plot in your write-up. What is the best value of k ? What is the lowest error?
 $k = 3$
 132.4

- (e) Explain your plot in (d) in terms of bias and variance. (In the definitions of *bias* and *variance*, you should think of the ground truth function g and the predicted hypothesis h as both being in the form of a longitude and a latitude, and you should assume that we integrate the bias-squared and the variance over the probability distribution of Spain travel photos that people might take.) In particular, given n training points, how is the bias different for $k = 1$ versus $k = n$? How is the variance different for $k = 1$ versus $k = n$? What happens for intermediate values of k ?
- (f) We do not need to weight every neighbor equally: closer neighbors may be more relevant. For this problem, weight each neighbor by the inverse of its distance (in feature space) to the test point by modifying `im2spain_starter.py`. Plot the error of k -NN regression with distance weighting vs. k , where k is the number of neighbors. What is the best value of k ? What is the MDE in miles? How does performance compare to part (e)?

$k=3$

131.0

Note: When computing the inverse distance, add a small value (e.g. 10^{-8}) to the denominator to avoid division by zero.

- (g) k -NN yields a *non-parametric* model which means its complexity can grow without bound as we increase the amount of training data. This is in contrast to *parametric* models such as linear regression that assume a fixed number of parameters, so the complexity of the model is bounded even if trained with infinite data. (We typically think of modern deep neural nets as functionally non-parametric, though they technically have a finite parameter size, because when we have more data we usually add more parameters.)

Let's compare the performance of k -NN with linear regression at different sizes of training datasets to get a sense of their respective "scaling curves". Plot the test error of both k -NN and linear regression for various percentages of training data. Which method would you expect to continue improving with twice as much training data?

kNN

Note: use the optimal value of k at each training dataset size by running grid search.

- (e) k from 1 to n : higher bias , lower variance
 for the intermediate values of k , at first bias is not that high,
 and variance decreases a lot , so MDE loss decreases . However , when k grows to a
 bigger value , higher variance leads to higher MDE loss on test set .